CSc 422, Program #1: Shared-Memory Parallel Programming

Due date: Thursday, March 6th, at 8:00am. **No late assignments will be accepted.**

This assignment has two purposes: (1) to introduce you to shared-memory parallel programming, and (2) to help you gain experience with computer science systems experimentation. You are required to do the following using the C programming language:

1. Write a columnsort function that sorts nonnegative integers in increasing order. The description of columnsort itself (from a paper from Dartmouth; hat tip to Dr. Lester McCann for the idea of using columnsort) is included in the gzipped tar file **here**.) Specifically, you will implement the following function:

   `void columnSort(int *A, int numThreads, int length, int width, double *elapsedTime).`

   For the sequential version of columnsort, which you should implement first, variable `numThreads` is never used. (I will pass in 1 for `numThreads`, but please note the sequential program does not use threads. This is merely to allow the same top-level function to be used for both the sequential and parallel versions.) For the sorting of columns (or rows, see below) in the odd-numbered steps, use the `qsort` library function (see `https://linux.die.net/man/3/qsort`).

   I will be writing a driver that will invoke function `columnSort`; as specified in the prototype, I will pass you a vector of integers as the first parameter. This length of the vector will always be a power of two. The third and fourth parameters are a length (denoted r hereafter) and width (denoted s hereafter) for you to use for the dimensions of the matrix described in the algorithm. (So, $r \times s$ is the total number of elements in A.) Through the arguments, you will be passing back the sorted vector of numbers and the elapsed time. You also may assume that the number of rows and columns are both powers of two.

   Please note that because C uses row-major storage, it is easier to actually do a "rowsort", though it is up to you. In rowsort, one simply uses an $s \times r$ matrix instead of an $r \times s$ one, but the idea is otherwise exactly the same as that of columnsort. (Note that if you do rowsort, then you are sorting rows [rather than columns] on all odd-numbered steps.) Either way, your program must use a dynamically sized two-dimensional array, which has been demonstrated in class (see the matrix multiplication example). Your code must be clearly documented, so we can figure out what you have done when we inspect your code.

2. Parallelize your columnsort program using `numThreads` threads. You should assume that s is much larger than the number of threads and that the number of threads will be a power of two. Create threads only once; you must use the dissemination barrier described in class (and no other barrier implementation) to ensure correctness. Note that you will likely need to use the `volatile` qualifier on the variable on which you are spinning in order to avoid having the optimizer generate incorrect code. (Specifically, one thread may be spinning on a register copy of a variable and another threads writes to memory to update it.)

3. Files `columnSort.h` and `driverColumnSort.c` are also in the gzipped tar file. You may not change `columnSort.h`.

Requirements for this program include:

1. You must comment your code so that we understand what you are doing.

2. There will need to be an initialization phase in which you copy the vector (passed in by the driver) into your $r \times s$ (or $s \times r$) matrix and a finalization phase in which you do the reverse. Do not parallelize either of these phases, and do not time them either.

3. You must use `gettimeofday` to measure time, just as in your sum program from the first assignment. Start the clock just before you create threads (which will be after the initialization phase), and stop the clock before the finalization phase.

4. You must include one of two makefiles (`Makefile-short` and `Makefile-long`) that will be provided to build your code. This means that you must name your files accordingly. Following are notes regarding the makefiles.

   - `make seqsort` will build sequential columnsort, and `make parsort` will build parallel columnsort.
   - File `Makefile-short` assumes that your sequential version is in file `seqColumnSort.c`, and the parallel version is in `threadColumnSort.c`.
   - File `Makefile-long` also assumes that your sequential version is in file `seqColumnSort.c`, and the parallel version is in `threadColumnSort.c`. It assumes, though, that (1) you have modularized the code and put routines common to the sequential and parallel versions in file `columnSortHelper.c` and that (2) its interface file is `columnSortHelper.h`.

Important note: If you simply sort the numbers we pass in any other way besides columnsort, it will be considered a violation of Academic Integrity. We will be reading your code, and you will be caught if you do this.

Perform speedup measurements on two and four cores within your Docker image. Note that speedup is computed relative to the *sequential* program, not the single-thread parallel program (i.e., T_s/T_p, where T_s is the time for the sequential program and T_p is the time for the parallel program on p cores). Use sizes 2^{20} through 2^{24}. The reported time for each test should be the median of three runs. (You can, if you like, write a script to run the tests.) The results you collect should be presented clearly as a table or graph. As part of your submission, include a file named `results.pdf` with the report and data. **You are not graded on how large your speedup is, but rather if your program is correct and well written.**

Submit your assignment to Gradescope (Prog 1); you must include the following files: `Makefile` (**you must rename whichever of `Makefile-short` and `Makefile-long` to Makefile**), `results.pdf`, `seqColumnSort.c` and `threadColumnSort.c`; and, if you are using the long Makefile, additionally `columnSortHelper.h` and `columnSortHelper.c`. You do not need to turn in `columnSort.h` or `driverColumnSort.c`.

Do not zip or tar files before turning them in, and do not turn in a directory. We will be compiling using `gcc -O2`, that is, optimization level two using `gcc`. No matter where you develop your program, it **must** work inside the Docker image.